INDIAN INSTITUTE OF TECHNOLOGY ROORKEE

Distributed Systems

Elementary Graph Algorithms

Distributed Spanning Tree Algorithms

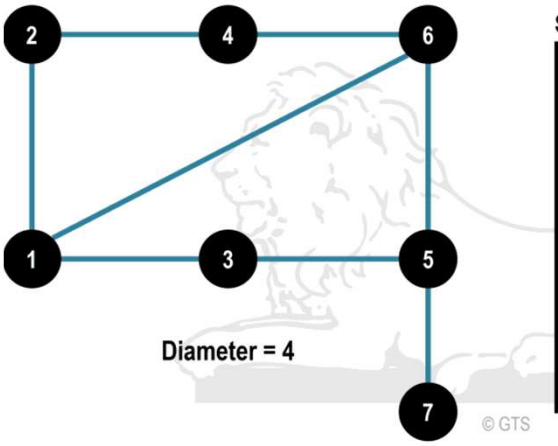
Design Issues:

- Each node has only a partial view of the graph (system), which is confined to its immediate neighbors
- A node can communicate with only its immediate neighbors along the incident edges.
- We assume unweighted undirected edges
- Communication is by message-passing on the edges
- The *diameter* of a graph is the minimum number of edges that need to be traversed to go from any node to any other node

  $\max_{i,j \in N}$ {length of the shortest path between i and j}

IIT ROORKEE

Diameter of a Graph



Shimbel Distance

v	1	2	3	4	5	6	7
1	0	1	1	2	2	1	3
2	1	0	2	1	3	2	4
3	1	2	0	3	1	2	2
4	2	1	3	0	2	1	3
5	2	3	1	2	0	1	1
6	1	2	2	1	1	0	2
7	3	4	2	3	1	2	0

Source: The Geography of Transport Systems

IIT ROORKEE

Synchronous single-initiator spanning tree algorithm using flooding

- The code for all processes is symmetrical and proceeds in rounds.
- Assume a designated root node, root, which initiates the algorithm
- The root initiates a flooding of QUERY messages in the graph to identify tree edges.
- The parent of a node is that node from which a QUERY is first received;
- If multiple QUERYs are received in the same round, one of the senders is randomly chosen as the parent

IIT ROORKEE

Synchronous single-initiator spanning tree algorithm using flooding

```
(local variables)
int visited, depth ⟵ 0
int parent ⟵ ⊥
set of int Neighbors ⟵ set of neighbors
(message types)
QUERY

(1)  if i = root then
(2)      visited ⟵ 1;
(3)      depth ⟵ 0;
(4)      send QUERY to Neighbors;
(5)  for round = 1 to diameter do
(6)      if visited = 0 then
(7)          if any QUERY messages arrive then
(8)              parent ⟵ randomly select a node from which
                    QUERY was received;
(9)              visited ⟵ 1;
(10)             depth ⟵ round;
(11)             send QUERY to Neighbors \ {senders of
                    QUERYs received in this round};
(12)     delete any QUERY messages that arrived in this round.
```

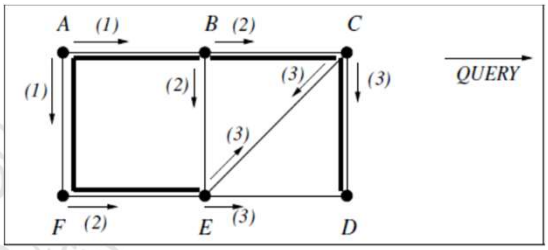
The pseudo-code for each process P_i

IIT ROORKEE

5

Synchronous single-initiator spanning tree algorithm using flooding

- Example



- Complexity
  - Local Space: O(degree)
  - Global Space: O(Σ local space)
  - Local Time: O(degree + diameter)
  - Message Time Complexity d rounds or message hops
  - Message Complexity: ≥ 1, ≤ 2 messages per edge => $[l, 2l]$
- Spanning Tree: analogous to breadth-first search

IIT ROORKEE

6

Asynchronous single-initiator spanning tree algorithm using flooding

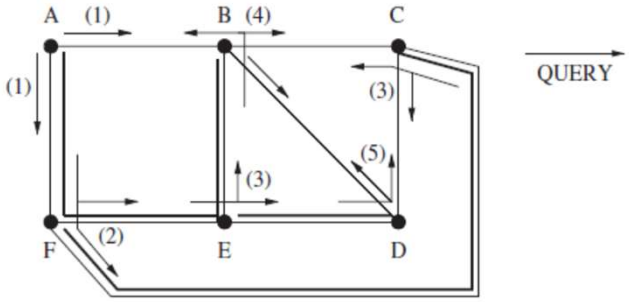
- Root initiates flooding of QUERY to identify tree edges
- parent: 1st node from which QUERY received
  - QUERY from parent replied to by ACCEPT
  - QUERY from non-parent replied to by REJECT
- Each node terminates its algorithm when it has received from all its non-parent neighbors a response to the QUERY sent to them
- There is no bound on the time it takes to propagate a message, and hence no notion of a message round

Asynchronous single-initiator spanning tree algorithm using flooding

```
(local variables)
int parent ← ⊥
set of int Children, Unrelated ← ∅
set of int Neighbors ← set of neighbors
(message types)
QUERY, ACCEPT, REJECT

(1) When the predesignated root node wants to initiate the algorithm:
(1a) if (i = root and parent = ⊥) then
(1b)     send QUERY to all neighbors;
(1c)     parent ← i.

(2) When QUERY arrives from j:
(2a) if parent = ⊥ then
(2b)     parent ← j;
(2c)     send ACCEPT to j;
(2d)     send QUERY to all neighbors except j;
(2e)     if (Children ∪ Unrelated) = (Neighbors \ {parent}) then
(2f)         terminate.
(2g) else send REJECT to j.

(3) When ACCEPT arrives from j:
(3a) Children ← Children ∪ {j};
(3b) if (Children ∪ Unrelated) = (Neighbors \ {parent}) then
(3c)     terminate.

(4) When REJECT arrives from j:
(4a) Unrelated ← Unrelated ∪ {j};
(4b) if (Children ∪ Unrelated) = (Neighbors \ {parent}) then
(4c)     terminate.
```

Asynchronous single-initiator spanning tree algorithm using flooding

- Example



This algorithm does not guarantee a breadth-first tree

Asynchronous single-initiator spanning tree algorithm using flooding

- Local termination: after receiving ACCEPT or REJECT from non-parent nbhs.
- Complexity:
  - Local space: O(degree)
  - Global space: O(Σ local space)
  - Local time: O(degree)
  - Message complexity: ≥ 2, ≤ 4 messages/edge => $[2l, 4l]$
- Message time complexity: d + 1 message hops, assuming synchronous communication
- Spanning tree: no claim can be made. Worst case height n - 1

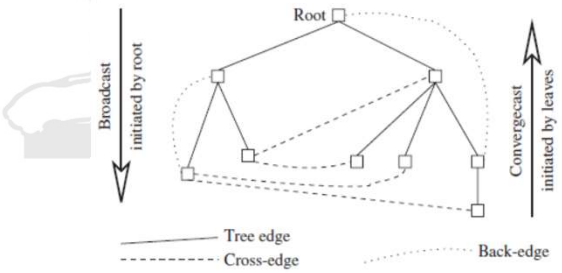
Broadcast algorithm on a spanning tree

BC1: The root sends the information to be broadcast to all its children.

Terminate.

BC2: When a (non-root) node receives information from its parent, it copies it and forwards it to its children.

Terminate.



IIT ROORKEE

11

Convergecast algorithm on a spanning tree

- It is initiated by the leaf nodes of the tree, usually in response to receiving a request sent by the root using a broadcast. The algorithm is specified as follows:

  CVC1: Leaf node sends its report to its parent. Terminate.

  CVC2: At a non-leaf node that is not the root: When a report is received from all the child nodes, the collective report is sent to the parent. Terminate.

  CVC3: At the root: When a report is received from all the child nodes, the global function is evaluated using the reports. Terminate.

IIT ROORKEE

12